

Avaliação Integradora I — encontros 0 a 5

Microcontroladores · DENE/UFMT · GABARITO

Nome: _____

1h30

Consulta permitida às folhas de referência do curso. Todas as questões falam do **mesmo sistema**, descrito abaixo; os dados e as fórmulas de que você precisa estão nas caixas. A resposta vale pelo raciocínio: um número certo sem justificativa vale metade.

O SISTEMA: UM PERFILÔMETRO DE ASFALTO

Um carro de passeio trafega a 100 km/h. Um sistema com o PIC18F4550 precisa medir a altura do asfalto **a cada 1 mm de estrada percorrida**, para mapear trincas e buracos. Ele tem duas entradas:

- um **encoder** de 100 pulsos por volta, preso a uma roda de raio 0,30 m, que entrega uma onda quadrada de 0 a 5 V em RC0 — o pino T13CKI do Timer1;
- um **sensor laser** de distância, que entrega em AN0 uma tensão de 0 a 5 V proporcional à distância até o asfalto, de 50 mm (0 V) a 150 mm (5 V).

O display do kit mostra a altura medida, e um LED em RC1 indica “medindo”.

DADOS

Núcleo a 16 MHz: $T_{CY} = 250$ ns
Conversor: 10 bits, 0 a 5 V, 1 LSB = 4,883 mV
Uma leitura do conversor: 15 μ s (60 ciclos)
Atualização do display: 1,3 ms (5200 ciclos)
Timer0 com divisor 1:8: $T0CON = 0x82$

FORMULÁRIO

Circunferência: $C = 2\pi r$ ($\pi \approx 3,1416$)
Distância por pulso: $d = C / \text{PPR}$
Velocidade: $v[\text{m/s}] = v[\text{km/h}] / 3,6$
Tempo por distância: $\Delta t = \Delta x / v$ $f = 1/T$
Ciclos: $n = \Delta t / T_{CY}$
Conversor: $V = \text{código} \cdot 5000 / 1024$ mV
Laser: $h = 50 + 100 \cdot V / 5$ mm (V em volts)
Timer0: $N = \Delta t / (T_{CY} \cdot \text{divisor})$, pré-carga = 65536 – N
Timer1 (16 bits): passo = $T_{CY} \cdot \text{divisor}$; estoura após 65536 passos

DISPLAY (HD44780)

Modo de 4 bits: cada byte vai em dois pulsos de E, pelas linhas D7–D4 — primeiro o nibble alto, depois o baixo. O controlador lê os dados na **borda de descida** de E. RS = 0: o byte é **instrução**. RS = 1: o byte é **dado** (caractere).

Caracteres: ‘0’ ... ‘9’ = 0x30 ... 0x39 ‘.’ = 0x2E
Instruções:

0x01 limpa a tela 0x80 | end posiciona o cursor
0x28 Function Set: 4 bits, 2 linhas
0x30–0x33 Function Set: 8 bits, 1 linha

Questão 1 (2,0 · encontros 0, 1 e 2)

Um colega escreveu a primeira versão do firmware. O LED em RC1 “medindo” deveria seguir o sinal do encoder, e nunca acende com o carro andando.

```
void main(void)
{
    LATC = 0x00;
    TRISC = 0x00;           /* "configura a porta C" */
    while (1) {
        LATCbits.LATC1 = PORTCbits.RC0; /* LED segue o encoder */
    }
}
```

- (a) O que `PORTCbits.RC0` devolve com esse código, e por quê?
- (b) Corrija a configuração com o mínimo de mudança: `TRISC = 0x01`;

RESPOSTA

(a) Sempre 0. $\text{TRISC} = 0x00$ põe todos os pinos da porta C como saída, inclusive RC0. O próprio microcontrolador força o pino ao valor do latch ($\text{LATC0} = 0$), e a leitura de PORTC devolve esse nível, qualquer que seja o encoder. Pior: a saída do encoder e a do microcontrolador ficam ligadas no mesmo fio, brigando.

(b) RC0 como entrada e o resto como saída: $\text{TRISC} = 0x01$;

CRITÉRIO

(a) 1,2: sempre 0 (0,4); RC0 virou saída e a leitura devolve o próprio latch (0,8). O conflito entre saídas, se citado, compensa falhas menores. (b) 0,8: RC0 como entrada, mantendo as saídas.

Questão 2 (2,0 · encontros 2 e o problema base)

(a) Que distância a roda percorre a cada pulso do encoder? A meta de uma amostra por milímetro é atendida só com esses pulsos?

(b) Numa caminhonete com rodas de raio 0,45 m, a distância por pulso aumenta ou diminui, e em que proporção?

RESPOSTA

(a) $C = 2 \cdot 3,1416 \cdot 0,30 = 1,885$ m, e $d = 1,885/100 = 18,85$ mm por pulso. Não atende: entre dois pulsos a roda anda quase 19 mm, e o sistema não sabe o que houve nos milímetros intermediários. Seria preciso um encoder de pelo menos 1885 pulsos por volta.

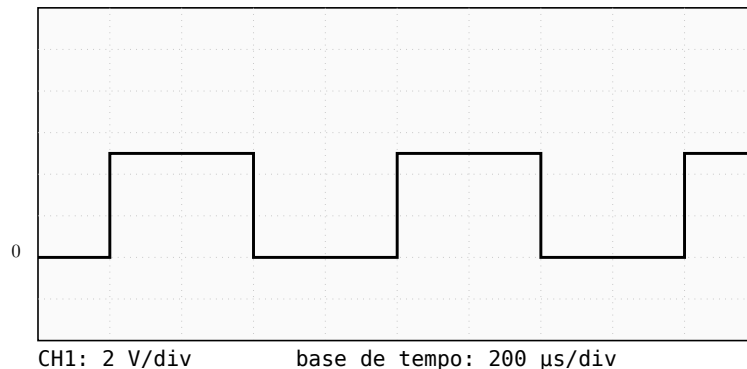
(b) Aumenta, na mesma proporção do raio: $d = 2\pi r / \text{PPR}$. Com 0,45 m, $d = 28,27$ mm — 1,5 vez pior.

CRITÉRIO

(a) 1,3: $d = 18,85$ mm, com a conta (0,8); não atende, com o motivo (0,5). (b) 0,7: aumenta, proporcional ao raio (0,4); 28,27 mm ou “1,5 vez” (0,3).

Questão 3 (2,0 · encontros 2 e 5)

Alguém mediu RC0 com o osciloscópio, com o carro andando:



(a) Leia na tela o período do sinal. Qual a frequência dos pulsos, e qual a velocidade do carro, em km/h? (Use o d da Questão 2.)

(b) Outra forma de medir a velocidade: o Timer1, com fonte interna, mede o **tempo entre dois pulsos** do encoder. O sistema precisa funcionar de **1 km/h a 150 km/h**. Quais divisores do Timer1 — 1:1, 1:2, 1:4 ou 1:8 — servem? Justifique com os dois extremos de velocidade.

RESPOSTA

(a) De uma borda de subida à seguinte são 4 divisões: $T = 4 \cdot 200 = 800$ μs, e $f = 1/800$ μs = 1250 Hz. Cada pulso vale 18,85 mm: $v = 1250 \cdot 0,01885 = 23,6$ m/s, ou ≈ 85 km/h. (A velocidade medida não precisa ser os 100 km/h do enunciado: é o carro naquele instante.)

(b) O período entre pulsos é d/v .

A **1 km/h** (0,278 m/s): $0,01885/0,278 = 67,9$ ms. O Timer1 estoura em $65536 \cdot 250$ ns · divisor: 16,4 ms (1:1), 32,8 ms (1:2), 65,5 ms (1:4) e 131 ms (1:8). Só com **1:8** o contador não estoura antes do pulso seguinte.

A **150 km/h** (41,7 m/s): $0,01885/41,7 = 452$ μs. Com 1:8, cada passo vale 2 μs, e o período cabe em **226 passos** — resolução de $\approx 0,4\%$, suficiente.

Resposta: só 1:8. O extremo lento decide o divisor mínimo; o extremo rápido confirma que ainda sobra resolução.

CRITÉRIO

(a) 1,0: período lido (0,4); frequência (0,3); velocidade (0,3). (b) 1,0: período a 1 km/h e os estouros, excluindo 1:1 a 1:4 (0,5); período a 150 km/h e a resolução com 1:8 (0,3); a conclusão, só 1:8 (0,2).

Questão 4 (2,0 · encontros 1, 2 e 5)

Suponha que o encoder foi trocado por um que dá **um pulso por milímetro**, e que a cada pulso o firmware lê o laser.

- (a) A 100 km/h, quanto tempo passa entre dois pulsos, e quantos ciclos de máquina isso dá?
- (b) O firmware consulta o indicador do Timer1 no laço, e o laço também atualiza o display. Quantos pulsos chegam durante uma atualização, e o que acontece com eles?
- (c) Para atender cada pulso no instante em que ele chega, o **hardware** precisaria desviar para uma rotina e depois voltar. Descreva o que ele precisa fazer — com o contador de programa e com os registradores que o laço usava — para que o laço continue exatamente de onde parou.

RESPOSTA

(a) $v = 100/3,6 = 27,78$ m/s; $\Delta t = 0,001/27,78 = 36$ μ s; $n = 36$ μ s / 250 ns = 144 ciclos — dos quais a leitura do conversor já consome 60.

(b) $1,3$ ms $\div 36$ μ s ≈ 36 pulsos. O indicador é **um bit**: sobe no primeiro, e os outros o encontram já em 1 — são perdidos, sem nenhum aviso. O mapa pula ≈ 35 mm de estrada a cada atualização de tela.

(c) Ao fim da instrução em curso, o hardware **guarda o PC** (o endereço da próxima instrução do laço) numa pilha e **carrega no PC um endereço fixo**, onde está a rotina. A rotina **preserva W, STATUS e BSR**, faz o trabalho curto e os restaura; uma instrução de **retorno** devolve ao PC o endereço guardado. Para o laço, nada foi pulado: só levou mais tempo.

CRITÉRIO

(a) 0,6: 36 μ s (0,3); 144 ciclos (0,3). (b) 0,6: ≈ 36 pulsos (0,2); indicador de um bit perde os outros, em silêncio (0,4). (c) 0,8: PC guardado e restaurado (0,4); endereço fixo (0,2); preservar registradores (0,2). Não se exige o nome “interrupção”.

Questão 5 (2,0 · encontros 3 e 4)

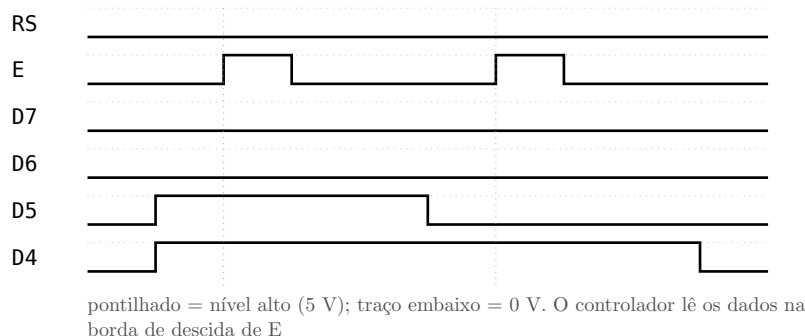
Numa amostra, o conversor devolveu o código **614** para o laser.

- (a) Qual a tensão, e qual a distância até o asfalto, em mm?
- (b) No firmware, a altura é calculada em centésimos de milímetro:

```
uint16_t cmm = 5000u + (codigo * 625u) / 64u;
```

A fórmula está certa, e o resultado sai errado para esse código. Por quê? Corrija.

(c) O primeiro caractere da altura vai para o display. A captura mostra as linhas do controlador durante esse envio, no modo de 4 bits:



Decodifique o byte enviado. O firmware tem um erro, visível na captura: qual é, e o que acontece com o display a partir daí? Use a caixa do display.

RESPOSTA

(a) $V = 614 \cdot 5000/1024 = 2998 \text{ mV} \approx 3,00 \text{ V}$, e $h = 50 + 100 \cdot 3,00/5 = 109,96 \text{ mm}$.

(b) O produto é calculado em 16 bits: $614 \cdot 625 = 383750$ não cabe em 65535, e sobra só o resto da divisão por 65536. Estoura a partir do código 105 — quase toda a faixa —, nada avisa, e o número que sai é plausível. Correção: produto em 32 bits, `5000u + ((uint32_t)codigo * 625u) / 64u` o resultado final (até 14 990) cabe em 16 bits, o intermediário não.

(c) Nas duas bordas de descida de E, D7–D4 valem `0011` e depois `0001`: o byte é `0x31`, o caractere `'1'` — o primeiro dígito de 109,96, correto.

O erro é **RS em 0** o tempo todo. Com $RS = 0$ o byte vai para o registrador de **instrução**, e `0x31` não é o caractere `'1'`: é o comando **Function Set** com $DL = 1$ — interface de **8 bits**, uma linha. O controlador passa a esperar um byte por pulso de E; o firmware continua mandando nibbles, e tudo o que vier depois sai lixo, até uma nova inicialização.

CRITÉRIO

(a) 0,6: tensão (0,2); altura (0,4). (b) 0,7: o produto estoura 16 bits (0,4); a correção em 32 bits (0,3). (c) 0,7: byte `0x31` decodificado (0,3); RS em 0, byte vai como instrução (0,2); Function Set em 8 bits e o lixo que se segue (0,2).